

SPEECH AND NATURAL LANGUAGE PROCESSING

Code: PEC-CS801C | Credit Points: 3 | Theory: 3 hrs/week

COMPREHENSIVE THEORY NOTES FOR SEMESTER EXAM

Unit 1	NLP Fundamentals – 6 Hours
Unit 2	Language Modelling with N-gram & Neural Networks – 7 Hours
Unit 3	Parts-of-Speech Tagging – 7 Hours
Unit 4	Syntactic & Statistical Parsing – 8 Hours
Unit 5	Vector Semantics & Word Embeddings – 8 Hours

1.1 Introduction to NLP

Natural Language Processing (NLP) is a subfield of Artificial Intelligence and linguistics that focuses on the interaction between computers and human languages. It enables computers to understand, interpret, manipulate, and generate natural language.

Definition

NLP is the ability of a computer program to understand human speech or text as it is spoken/written. It combines computational linguistics with statistical, machine learning, and deep learning models.

Applications of NLP

- Machine Translation (e.g., Google Translate)
- Sentiment Analysis (opinion mining)
- Speech Recognition (e.g., Siri, Alexa)
- Chatbots and Conversational Agents
- Text Summarization
- Question Answering Systems
- Information Retrieval and Search Engines
- Spam Detection in emails
- Named Entity Recognition (NER)
- Text Classification and Categorization

 **Exam Tip:** Applications are frequently asked in 2-mark and 5-mark questions. Memorize at least 8 applications with brief examples.

1.2 NLP Phases (Pipeline)

NLP processes text through a series of levels/phases, each building upon the previous:

- **Phase 1: Morphological Analysis**
 - Analyzes the structure of words
 - Deals with morphemes (smallest meaningful units)
 - Example: 'running' → 'run' + 'ning'
- **Phase 2: Lexical Analysis**
 - Tokenization – breaking text into words/tokens
 - Identifies parts of speech for each word
 - Handles stop words, stemming, lemmatization
- **Phase 3: Syntactic Analysis (Parsing)**
 - Checks grammatical structure of sentences
 - Creates parse trees showing sentence structure
 - Ensures words are arranged in grammatically valid ways
- **Phase 4: Semantic Analysis**

- Determines the meaning of the text
- Resolves ambiguities in word meanings
- Word Sense Disambiguation (WSD)
- **Phase 5: Discourse Integration**
 - Interprets meaning across sentences
 - Handles pronoun resolution (co-reference)
 - Understands context from prior sentences
- **Phase 6: Pragmatic Analysis**
 - Understands the intent and real-world meaning
 - Considers context, tone, and situational knowledge
 - Example: 'Can you pass the salt?' is a request, not a yes/no question

★ **Key Point:** The NLP pipeline flows: Morphological → Lexical → Syntactic → Semantic → Discourse → Pragmatic. Each phase adds a deeper level of understanding.

1.3 Difficulty/Challenges of NLP

NLP is difficult because of the inherent properties of natural language:

Types of Ambiguity

- Lexical Ambiguity – A word has multiple meanings. Example: 'bank' (river bank or financial bank)
- Structural/Syntactic Ambiguity – A sentence can be parsed in multiple ways. Example: 'I saw the man with the telescope' (who has the telescope?)
- Semantic Ambiguity – Multiple possible meanings for a sentence even after parsing
- Referential Ambiguity – Unclear what a pronoun or phrase refers to. Example: 'John told Paul he was wrong' (who is 'he'?)
- Pragmatic Ambiguity – The speaker's intent is unclear from the literal meaning

Other Challenges

- Idioms and Figurative Language – 'It's raining cats and dogs'
- Sarcasm and Irony – Hard to detect computationally
- Domain-specific Language – Medical, legal jargon
- Spelling and Grammatical Errors
- Informal Language, Slang, and Abbreviations
- Multilingual and Code-switching challenges

1.4 Spelling Error and Noisy Channel Model

Types of Spelling Errors

- Non-word Errors – 'graffe' instead of 'giraffe' (not a valid word)
- Real-word Errors – 'their' instead of 'there' (valid word but wrong context)
- Cognitive Errors – Homophones, e.g., 'piece' vs 'peace'

Noisy Channel Model

The Noisy Channel Model treats spelling correction as a probabilistic inference problem.

★ **Noisy Channel Formula:** $P(w|x) \propto P(x|w) \times P(w)$ Where: w = correct word, x = observed (misspelled) word, $P(x|w)$ = channel/error model (probability of making error), $P(w)$ = language model (probability of the word in context)

- The model finds the most likely intended word given the observed misspelling
- Error model captures how likely a correct word is corrupted to the observed word
- Language model captures how likely a word is to appear in that context
- Combines edit distance with probability to rank candidate corrections

Edit Distance

- Minimum Levenshtein Distance – minimum number of single-character edits (insertion, deletion, substitution) to transform one word into another
- Example: 'kitten' → 'sitting' requires 3 edits
- Used to generate candidate corrections for misspelled words

✍ **Exam Tip:** Noisy Channel Model formula $P(w|x) \propto P(x|w) \times P(w)$ is very commonly asked. Know what each component means.

1.5 Parts of Speech (POS)

Parts of Speech are categories of words based on their grammatical role in a sentence.

POS Tag	Description	Example
Noun (NN)	Person, place, thing, idea	<i>dog, city, happiness</i>
Verb (VB)	Action or state of being	<i>run, is, think</i>
Adjective (JJ)	Describes a noun	<i>beautiful, large</i>
Adverb (RB)	Modifies verbs/adjectives	<i>quickly, very</i>
Pronoun (PRP)	Replaces a noun	<i>he, she, it, they</i>
Preposition (IN)	Shows relationship	<i>in, on, at, over</i>
Conjunction (CC)	Connects words/clauses	<i>and, but, or</i>
Determiner (DT)	Limits or defines a noun	<i>the, a, an, this</i>
Interjection (UH)	Expresses emotion	<i>Oh!, Wow!, Hey!</i>

1.6 Formal Grammar of English

Formal grammars provide a systematic set of rules to describe the structure of sentences.

Context-Free Grammar (CFG)

- A CFG consists of: Terminal symbols (words), Non-terminal symbols (S, NP, VP), Production rules, Start symbol (S)
- Example rules: $S \rightarrow NP VP$ | $NP \rightarrow Det N$ | $VP \rightarrow V NP$ | $Det \rightarrow 'the'$ | $N \rightarrow 'dog'$
- A sentence is valid if it can be derived using the rules

Chomsky Hierarchy

- Type 0: Unrestricted Grammars (Turing machines)
- Type 1: Context-Sensitive Grammars
- Type 2: Context-Free Grammars (most programming languages, NLP parsing)
- Type 3: Regular Grammars (finite automata, regex)

 **Exam Tip:** Know CFG production rules format and the Chomsky Hierarchy levels for exams. Be able to derive a simple sentence using CFG rules.

2.1 Language Models

A Language Model (LM) assigns probability to a sequence of words. It answers: 'How likely is this sentence?'

★ **Purpose:** Language models help predict the next word in a sequence and evaluate the probability of an entire sentence. They are fundamental to speech recognition, machine translation, spelling correction, and text generation.

2.2 N-gram Language Models

What is an N-gram?

- A contiguous sequence of N words from a given text
- Unigram (N=1): Individual words – $P(w_1) \times P(w_2) \times \dots \times P(w_n)$
- Bigram (N=2): Consecutive word pairs – approximates using $P(w_i | w_{i-1})$
- Trigram (N=3): Consecutive word triples – $P(w_i | w_{i-2}, w_{i-1})$

Markov Assumption

The N-gram model relies on the Markov assumption: the probability of a word depends only on the previous N-1 words, not the entire history.

★ **Bigram Formula:** $P(w_1, w_2, \dots, w_n) \approx \prod P(w_i | w_{i-1})$ Count-based: $P(w_i | w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$

Example

Sentence: 'I love natural language processing'

- Bigrams: (I,love), (love,natural), (natural,language), (language,processing)
- $P(\text{processing}|\text{language}) = \text{Count}(\text{language processing}) / \text{Count}(\text{language})$

2.3 Smoothing Techniques

Problem: Zero probability for unseen word sequences (data sparsity problem).

Smoothing assigns small probabilities to unseen events. Key techniques:

Add-One (Laplace) Smoothing

★ **Formula:** $P_{\text{laplace}}(w_i | w_{i-1}) = [\text{Count}(w_{i-1}, w_i) + 1] / [\text{Count}(w_{i-1}) + V]$ where V = vocabulary size

- Adds 1 to all counts, including unseen bigrams
- Drawback: Over-smooths; assigns too much probability mass to unseen events

Add-k Smoothing

- Generalizes Laplace by adding a fraction k ($0 < k < 1$) instead of 1

★ **Formula:** $P_{\text{add-k}}(w_i|w_{i-1}) = [\text{Count}(w_{i-1}, w_i) + k] / [\text{Count}(w_{i-1}) + k \times V]$

Backoff and Interpolation

- Backoff: Use trigram if available; back off to bigram if count is zero; further to unigram
- Interpolation: Mix probabilities from all N-gram orders

★ **Interpolation Formula:** $P(w_i|w_{i-2}, w_{i-1}) = \lambda_1 \times P(w_i|w_{i-2}, w_{i-1}) + \lambda_2 \times P(w_i|w_{i-1}) + \lambda_3 \times P(w_i)$
where $\lambda_1 + \lambda_2 + \lambda_3 = 1$

2.4 Evaluating Language Models – Perplexity

Perplexity measures how well a language model predicts a test corpus. Lower perplexity = better model.

★ **Perplexity Formula:** $PP(W) = P(w_1, w_2, \dots, w_N)^{-1/N} = N\text{-th root of } 1/P(w_1, \dots, w_N)$
Equivalently: $PP(W) = 2^H(W)$ where H is entropy

- A model with perplexity k is as uncertain as if it had k equally likely choices at each step
- A good language model has low perplexity on test data

🔪 **Exam Tip:** Perplexity formula and its interpretation (lower = better) is almost always tested. Understand the inverse probability relationship.

2.5 Neural Network Basics for NLP

Feedforward Neural Network

- Input layer: word embeddings or feature vectors
- Hidden layers: learn non-linear representations
- Output layer: probability distribution over vocabulary (using Softmax)
- Activation functions: Sigmoid, ReLU, Tanh

Training Neural Networks

- Forward Pass: compute predictions
- Loss Function: Cross-entropy loss for classification
- Backpropagation: compute gradients using chain rule
- Gradient Descent: update weights to minimize loss
- Hyperparameters: learning rate, batch size, epochs, hidden units

2.6 Neural Language Models

Why Neural LMs over N-grams?

- N-grams suffer from data sparsity and cannot generalize well
- Neural LMs represent words as dense vectors (embeddings) that capture semantic similarity
- Similar words have similar representations, enabling generalization

Recurrent Neural Network (RNN) Language Model

- Processes text sequentially, maintaining a hidden state
- Hidden state $h_t = f(h_{t-1}, x_t)$ – captures context from all previous words
- Output: probability distribution over next word
- Problem: Vanishing gradient for long sequences

LSTM (Long Short-Term Memory)

- Addresses vanishing gradient through gating mechanisms
- Forget gate: decides what to discard from cell state
- Input gate: decides what new info to add to cell state
- Output gate: decides what to output
- Cell state carries long-range dependencies

Case Study: Neural LM in NLP Systems

- Speech Recognition: Predict likely next word to improve accuracy
- Machine Translation: Encoder-decoder LSTM/Transformer architectures
- Text Generation: GPT-style autoregressive models
- BERT: Bidirectional LM for understanding context from both directions

🔪 **Exam Tip:** Understand N-gram vs Neural LM tradeoffs, the concept of word embeddings, and why LSTM solves vanishing gradient.

3.1 Introduction to POS Tagging

POS Tagging is the process of assigning a part-of-speech label (noun, verb, adjective, etc.) to each word in a sentence based on its definition and context.

★ **Example:** 'The cat sat on the mat' → The/DT cat/NN sat/VBD on/IN the/DT mat/NN

Why is POS Tagging Hard?

- Many words are ambiguous: 'book' can be NN (a book) or VB (to book a ticket)
- Context determines the correct tag
- Tagset ambiguity: different tagsets (Penn Treebank has 45 tags)

3.2 Tagsets

Penn Treebank Tagset (most common)

- NN – Noun singular, NNS – Noun plural, NNP – Proper noun
- VB – Verb base form, VBD – Past tense, VBG – Gerund, VBN – Past participle, VBP – Non-3rd person singular, VBZ – 3rd person singular
- JJ – Adjective, JJR – Comparative, JJS – Superlative
- RB – Adverb, RBR – Comparative, RBS – Superlative
- DT – Determiner, IN – Preposition, PRP – Personal pronoun, CC – Conjunction
- CD – Cardinal number, POS – Possessive ending (e.g., 's)

3.3 Early Approaches to POS Tagging

Rule-Based Tagging

- Assigns tags using hand-crafted linguistic rules
- Uses large dictionaries and rules like: 'If word ends in -ing, tag as VBG'
- Brill Tagger is a famous rule-based tagger
- Advantage: Interpretable, transparent
- Disadvantage: Requires extensive manual rule creation, brittle

Transformation-Based Learning (TBL) – Brill Tagger

- Starts with initial tags assigned by simple baseline
- Learns transformation rules: 'Change tag X to Y if condition Z holds'
- Rules are applied iteratively to improve accuracy
- Example rule: Change NN to VB if previous word is MD (modal verb)
- Combines rule-based interpretability with learning from data

✎ **Exam Tip:** TBL/Brill Tagger is frequently asked: explain the transformation rule format and the iterative learning process.

3.4 POS Tagging using HMM

Hidden Markov Models treat POS tagging as a sequence labeling problem where the tags are 'hidden states' and words are 'observations'.

HMM Components

- States (Q): POS tags – {NN, VB, DT, ...}
- Observations (O): Words in the sentence
- Transition Probabilities A: $P(t_i | t_{i-1})$ – probability of tag t_i following tag t_{i-1}
- Emission/Observation Probabilities B: $P(w_i | t_i)$ – probability of word w_i given tag t_i
- Initial State Probabilities π : $P(t_1)$ – probability of being in state t_1 at start

★ **HMM Tagging Goal:** Find the sequence of tags T^* that maximizes: $T^* = \operatorname{argmax}_T P(T|W)$
 $= \operatorname{argmax}_T P(W|T) \times P(T)$

Viterbi Algorithm

- Dynamic programming algorithm to find the most likely tag sequence
- $\text{viterbi}[t][j] = \text{max probability of being in state } j \text{ at time } t$
- $\text{viterbi}[t][j] = \max_i [\text{viterbi}[t-1][i] \times A(i,j)] \times B(j, w_t)$
- Backtracking recovers the best sequence of tags

Training HMM

- Transition probabilities: $\text{Count}(t_{i-1}, t_i) / \text{Count}(t_{i-1})$
- Emission probabilities: $\text{Count}(t_i, w_i) / \text{Count}(t_i)$
- Trained on labeled corpora (tagged text)

✎ **Exam Tip:** Viterbi algorithm is a MUST-KNOW topic. Be able to fill in a Viterbi table step by step with given probabilities. Practice numerical examples.

3.5 Neural Models for POS Tagging

Why Neural Models?

- Can learn features automatically from data
- Capture long-range dependencies better
- State-of-the-art accuracy

Bidirectional LSTM (BiLSTM) for POS Tagging

- Input: Word embeddings for each word

- Forward LSTM: processes sequence left to right
- Backward LSTM: processes sequence right to left
- Concatenate forward and backward hidden states for each word
- Feed to a softmax layer to predict POS tag
- Benefits: Captures both left and right context of each word

Transformer-based POS Tagging (BERT)

- Pre-trained on large text corpora
- Fine-tuned on labeled POS data
- Achieves near-human accuracy on standard benchmarks

4.1 Introduction to Parsing

Parsing is the process of analyzing a sentence to determine its grammatical structure according to a given formal grammar. The result is a parse tree or derivation tree.

4.2 Top-Down Parsing

- Starts from the root symbol (S) and applies grammar rules to expand non-terminals
- Works left-to-right, trying to match the input
- Uses prediction: predicts what words should appear based on grammar rules
- Problem: Left-recursive grammars cause infinite loops
- Example: Start with $S \rightarrow NP VP$, expand $NP \rightarrow Det N$, match 'the', 'dog'

★ **Top-Down:** Strategy: Start with S, expand rules until you reach terminal symbols (words).
Direction: Grammar $\rightarrow$ Words. Problem: Backtracks unnecessarily when rules don't match.

4.3 Bottom-Up Parsing

- Starts from the input words (leaves) and applies grammar rules in reverse
- Combines terminals into non-terminals, working towards the root S
- Uses completion: combines recognized phrases into larger phrases
- More efficient for handling left-recursive grammars
- Shift-Reduce parsing is a common bottom-up algorithm

★ **Bottom-Up:** Strategy: Start with words, apply rules in reverse until you reduce to S.
Direction: Words $\rightarrow$ Grammar. Better for left-recursive grammars.

4.4 Treebank

- A treebank is a corpus of parsed sentences with annotated syntactic structure
- Penn Treebank is the most well-known English treebank (~1 million words)
- Used to train statistical and neural parsers
- Contains phrase structure trees with POS tags
- Example annotation: (S (NP (DT The) (NN dog)) (VP (VBD sat)))

4.5 CKY Parsing (Cocke-Kasami-Younger)

CKY is a dynamic programming algorithm for parsing Context-Free Grammars in Chomsky Normal Form (CNF).

Chomsky Normal Form (CNF)

★ **CNF Rules:** Every production must be in one of two forms: 1. $A \rightarrow B C$ (non-terminal $\rightarrow$ two non-terminals) 2. $A \rightarrow a$ (non-terminal $\rightarrow$ one terminal) No empty productions (except possibly $S \rightarrow \epsilon$)

CNF Conversion Steps

- Step 1: Remove unit productions ($A \rightarrow B$)
- Step 2: Remove epsilon productions
- Step 3: Binarize rules with more than 2 symbols on right-hand side
- Step 4: Replace terminals in rules with more than one symbol on right

CKY Algorithm

- Builds a parse table (chart) of size $n \times n$ for a sentence of n words
- $\text{table}[i][j]$ = set of non-terminals that can generate words i to j
- Fill table bottom-up from length-1 spans to length- n span
- For each cell $[i][j]$ and each split point k : if $A \rightarrow BC$, B spans $[i,k]$ and C spans $[k+1,j]$, then A spans $[i,j]$
- If S is in $\text{table}[1][n]$, the sentence is grammatical

✎ **Exam Tip:** CKY parsing is a very important exam topic. Be able to: (1) convert a grammar to CNF, (2) fill in the CKY parse table step by step for a given sentence and grammar.

4.6 Statistical Parsing

Statistical parsers use probabilities to select the most likely parse tree from all possible parses.

Probabilistic Context-Free Grammar (PCFG)

- A CFG augmented with probabilities for each production rule
- Each rule $A \rightarrow \alpha$ has a probability $P(A \rightarrow \alpha)$
- For a given non-terminal A , all rule probabilities must sum to 1

★ **PCFG Formula:** $P(T) = \prod P(A \rightarrow \alpha)$ for all rules used in tree T Best parse: $T^* = \text{argmax}_T P(T|S)$

PCFG Training

- Read off rule probabilities from treebank counts
- $P(A \rightarrow \alpha) = \text{Count}(A \rightarrow \alpha) / \text{Count}(A)$
- Example: If NP appears 100 times and $\text{NP} \rightarrow \text{Det N}$ appears 60 times, then $P(\text{NP} \rightarrow \text{Det N}) = 0.6$

4.7 Probabilistic CKY Parsing

- Extends CKY to use PCFG probabilities

- $\text{table}[i][j][A]$ = maximum probability that A generates words i to j
- $\text{table}[i][j][A] = \max \text{ over all rules } A \rightarrow BC \text{ and split points } k \text{ of: } P(A \rightarrow BC) \times \text{table}[i][k][B] \times \text{table}[k+1][j][C]$
- Uses backpointer table to reconstruct the best parse tree
- This is the Viterbi parse (most probable parse)

 **Exam Tip:** Probabilistic CKY = CKY + PCFG probabilities. Know how to compute probabilities in the table and find the most probable parse. Often asked for 10-15 marks.

5.1 Vector Semantics

Vector Semantics represents the meaning of words as points (vectors) in a high-dimensional space, such that words with similar meanings are located near each other.

★ **Distributional Hypothesis:** 'You shall know a word by the company it keeps' (Firth, 1957). Words that appear in similar contexts tend to have similar meanings.

5.2 Words and Vectors

Term-Document Matrix

- Rows = words (terms), Columns = documents
- Cell value = frequency of word in document
- Used in Information Retrieval

Term-Term (Word-Word) Matrix

- Rows and columns are both words
- Cell value = how often words co-occur within a context window
- Context window: ± 2 words surrounding the target word
- Results in a co-occurrence matrix

5.3 Measuring Similarity

Cosine Similarity

★ **Formula:** $\text{cosine}(v, w) = (v \cdot w) / (|v| \times |w|) = \sum(v_i \times w_i) / [\sqrt{\sum v_i^2} \times \sqrt{\sum w_i^2}]$ Range: -1 (opposite) to 1 (identical), 0 = orthogonal (unrelated)

- Measures the angle between two vectors (ignores magnitude)
- Better than Euclidean distance for text since it handles different document lengths
- Cosine similarity of 1 = identical direction (very similar words)

TF-IDF Weighting

★ **Formula:** $\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$ $\text{TF}(t, d) = \text{count}(t \text{ in } d) / \text{total words in } d$ $\text{IDF}(t) = \log(N / \text{df}(t))$ where N = total docs, $\text{df}(t)$ = docs containing t

- TF (Term Frequency): How often a term appears in a document
- IDF (Inverse Document Frequency): Penalizes common words like 'the'
- High TF-IDF = word is frequent in document but rare across corpus = informative

Pointwise Mutual Information (PMI)

★ **Formula:** $PMI(w,c) = \log_2 [P(w,c) / (P(w) \times P(c))]$ $PPMI = \max(PMI, 0)$ (Positive PMI discards negative values)

- Measures how much more often words co-occur than expected by chance
- High PMI = strong association between words
- PPMI is more commonly used (treats negative PMI as 0)

✍ **Exam Tip:** Know cosine similarity formula AND how to calculate it. TF-IDF and PPMI formulas are also commonly asked. Practice numerical examples.

5.4 Semantics with Dense Vectors

Instead of sparse, high-dimensional co-occurrence vectors, dense vectors (word embeddings) pack semantic information into short (50–300 dimensions) dense vectors.

- Sparse vectors: Most values are 0, very high-dimensional ($|\text{vocabulary}|$)
- Dense vectors: All values are non-zero, low-dimensional (50–300)
- Dense vectors generalize better and are computationally efficient

5.5 SVD and Latent Semantic Analysis (LSA)

Singular Value Decomposition (SVD)

★ **SVD Formula:** $A = U \Sigma V^T$ where A = original term-document matrix, U = left singular vectors (words), Σ = diagonal matrix of singular values, V^T = right singular vectors (documents)

- Factorizes a matrix into three matrices
- Captures latent (hidden) semantic relationships

LSA (Latent Semantic Analysis)

- Apply SVD to term-document matrix
- Truncate to k dimensions (keep only top- k singular values)
- This gives low-dimensional dense vector representation for words and documents
- Words used in similar contexts get similar vectors even without direct co-occurrence
- Applications: Document similarity, Information Retrieval, synonym detection

5.6 Word2Vec: Embeddings from Prediction

Word2Vec learns word embeddings by training a neural network to predict context words from target words (or vice versa).

Skip-gram Model

★ **Skip-gram Goal:** Given a center/target word, predict the surrounding context words within a window. Maximize: $\sum \log P(\text{context word} \mid \text{center word})$

- Input: one-hot vector of target word
- Hidden layer: word embedding matrix W ($|V| \times d$)
- Output: probability over all vocabulary words (softmax)
- After training, the rows of W are the word embeddings
- Better for infrequent words; works well for small datasets

CBOW (Continuous Bag of Words)

★ **CBOW Goal:** Given the surrounding context words, predict the center/target word. Maximize: $\sum \log P(\text{center word} \mid \text{context words})$

- Input: average of context word vectors
- Output: predicts the target word
- Faster to train; works well for frequent words and larger datasets

Skip-gram vs CBOW Comparison

Feature	Skip-gram	CBOW
Task	Predict context from word	Predict word from context
Speed	Slower training	Faster training
Rare words	Better performance	Weaker performance
Data size	Works with small data	Better with large data

Word2Vec Properties

- Captures analogical relationships: king - man + woman $\approx$ queen
- Similar words cluster together in embedding space
- Negative sampling speeds up training
- Typical dimensions: 100–300

🔪 **Exam Tip:** Skip-gram vs CBOW comparison is a guaranteed exam question. Know the objective function of each, direction of prediction, and tradeoffs.

5.7 Concept of Word Sense

- A single word can have multiple senses/meanings – called polysemy
- Example: 'bank' can mean financial institution or river bank
- Word Sense Disambiguation (WSD): determine which sense is meant in context
- Monosemous words: only one sense (e.g., 'photosynthesis')
- Homonyms: different meanings, unrelated (e.g., 'bat' = animal or cricket bat)
- Polysemy: related senses of same word (e.g., 'head' of person, head of organization)

WSD Approaches

- Knowledge-based: Use WordNet sense definitions and relations
- Supervised: Train classifier on sense-labeled data
- Unsupervised: Word sense induction without labeled data

5.8 Introduction to WordNet

- WordNet is a large lexical database of English words
- Organizes words into synsets (sets of synonyms representing a concept)
- Example: {car, automobile, auto} form a synset

WordNet Relations

- Synonymy: Same meaning – {car, automobile}
- Hypernymy (IS-A): More general term – 'dog' IS-A 'animal'
- Hyponymy: More specific term – 'poodle' IS-A 'dog'
- Meronymy (PART-OF): 'wheel' is PART-OF 'car'
- Antonymy: Opposite meaning – 'hot' ↔ 'cold'

★ **WordNet Uses:** WSD (using gloss overlap), Information Retrieval (query expansion), NLP evaluation, computing semantic similarity between concepts using path length in WordNet hierarchy.

🔪 **Exam Tip:** WordNet relations (IS-A, PART-OF, synonymy, antonymy) and how WordNet is used for WSD are commonly asked. Memorize the relation types with examples.

QUICK REVISION – KEY FORMULAS & CONCEPTS

Topic	Key Formula / Concept
Noisy Channel	$P(w x) \propto P(x w) \times P(w)$ — combine error model and language model
Bigram Probability	$P(w_i w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$
Laplace Smoothing	$P(w_i w_{i-1}) = [C(w_{i-1}, w_i) + 1] / [C(w_{i-1}) + V]$
Interpolation	$P(w_i w_{i-2}, w_{i-1}) = \lambda_1 P_3 + \lambda_2 P_2 + \lambda_3 P_1$, where $\lambda_1 + \lambda_2 + \lambda_3 = 1$
Perplexity	$PP(W) = P(w_1 \dots w_N)^{-1/N}$ — lower is better
HMM Tagging	$T^* = \text{argmax}_T P(W T) \times P(T)$ using Viterbi algorithm
Viterbi	$vt[j] = \max_i [v(t-1)[i] \times A(i, j)] \times B(j, wt)$
PCFG Parse	$P(T) = \text{product of all rule probabilities used}$
PCFG Estimation	$P(A \rightarrow \alpha) = \text{Count}(A \rightarrow \alpha) / \text{Count}(A)$
Cosine Similarity	$\cos(v, w) = (v \cdot w) / (v w)$ — range -1 to 1
TF-IDF	$\text{TF-IDF} = \text{TF} \times \log(N/\text{df})$ — rewards informative terms
PPMI	$\text{PPMI}(w, c) = \max(\log_2[P(w, c)/(P(w)P(c))], 0)$
SVD/LSA	$A = U\Sigma V^T$ — factorize for dense semantic vectors
Skip-gram	Predict context words from center word
CBOW	Predict center word from context words
CNF Rules	$A \rightarrow BC$ (two non-terminals) or $A \rightarrow a$ (one terminal)

Best of luck for your exam! 🎓